Evaluating GPT-5.4-mini for Gherkin Test Generation: An Empirical Study

Nguyen Minh Hoang[1], Nguyen Hieu An[1], and Le Thi Quynh Chi[1*]

Department of Software Engineering, FPT University, Ho Chi Minh City, Vietnam
Email: hoangminh6112006@gmail.com (N.M.H.); hieuannguyen2k6@gmail.com (N.H.A.); chiltq6@fe.edu.vn (L.T.Q.C.)
[*] Corresponding author

Abstract. Large language models can draft behavior-driven development (BDD) scenarios, but valid syntax does not guarantee preserved business intent. We evaluate GPT-5.4-mini on reconstructing 261 project-maintained Gherkin files from Sylius, Apache Fineract and Diaspora from user stories reverse-engineered from them, with one same-project example per prompt. We measure parser compliance with behave and skeleton cosine similarity to the reference. Excluding the three prompt exemplars ($N = 258$), 255 files (98.8%, 95% CI [96.6%, 99.8%]) are valid Gherkin, above the 80% threshold ($p < 0.001$). The median similarity is 0.853; a one-sided exact sign test does not show that it exceeds the proposal's 0.85 reference point ($p = 0.209$, $\delta = 0.054$). Only the e-commerce project exceeds it; the decision changes at 0.84. In an exploratory audit of 60 high-similarity files, 25 contain a business-logic defect (AI-drafted, human-reviewed labels; $\kappa = 0.571$ and 0.609 against an independent rater), and seeded defects barely change the score. In a same-time rerun, removing the example lowers the median to 0.811, while another exemplar moves single files in both directions with no detectable average effect. Within this reconstruction setting, GPT-5.4-mini is a drafting aid whose business logic needs human review, and embedding similarity alone cannot certify generated Gherkin.

Keywords: Large language models, Gherkin, Behavior-driven development, Acceptance testing, Software testing, GPT-5.4-mini

1 Introduction

Behavior-driven development expresses expected software behavior in executable or executable-adjacent scenarios, commonly using Gherkin. Its cost is not limited to writing Given–When–Then syntax: engineers must preserve domain preconditions, project conventions, data dependencies and expected outcomes while keeping feature files parsable by the project toolchain. In the three projects of this study, a Sylius checkout scenario may depend on channel, country, shipping-method state and confirmation behavior, Apache Fineract scenarios encode dates, repayment periods, balances and data tables, and Diaspora scenarios

describe browser-visible behavior such as navigation, tags, comments and notifications. A generated scenario can therefore be syntactically valid yet incomplete as a specification of business behavior, and converting a compact Connextra story into project-consistent Gherkin is a constrained generation problem rather than a simple paraphrasing task.

1.1 Research Gap and Motivation

Prior work shows that LLMs can generate BDD artifacts [1,2,3,4], but the evaluations differ substantially in data scale, model family and validation method (Section 2). These studies establish feasibility, but they do not jointly answer whether a current compact model can produce both parser-ready and structurally close Gherkin across multiple public software domains. In particular, none of the five studies in Table S5 of the supplementary material tests whether its semantic measure reacts to business-logic defects, and none varies the example given in the prompt.

This study addresses that narrower gap by evaluating the pinned gpt-5.4-mini-2026-03-17 snapshot on 261 stories from three unrelated open-source projects, using same-project one-shot prompting and two separate outcome measures. The contribution is not a claim that GPT-5.4-mini is superior to earlier models, but an auditable evaluation design that separates syntactic validity from structural similarity on a public, multi-domain benchmark and tests how far the similarity metric can be trusted. Because the stories were reverse-engineered from the reference files and the 0.85 reference point was not calibrated, this study therefore includes a leakage measurement, a defect audit, a perturbation test, a reference-point sensitivity analysis, per-project results, a prompt ablation and a same-time rerun.

1.2 Research Questions and Contributions

RQ1 – Structural skeleton resemblance. When GPT-5.4-mini reconstructs feature files from user stories reverse-engineered from them, how is the skeleton cosine similarity to the reference distributed around the reference point of 0.85 fixed in the proposal?
RQ2 – Syntactic validity. Does GPT-5.4-mini generate feature files that the behave Gherkin parser accepts at a rate above 80%?

The paper makes three contributions: (1) a 261-case, multi-domain benchmark derived from project-maintained reference files in Sylius, Apache Fineract, and Diaspora, with its construction and leakage risk documented; (2) an evaluation that reports parser compliance separately from structural similarity and checks the similarity metric against defect labels, seeded defects and alternative reference points; and (3) a replication package containing the pinned model identifier, prompts, scripts, API logs, outputs, and all additional analyses.

Section 2 reviews related work. Section 3 describes the dataset, the generation pipeline, the metrics and the analyses, and Section 4 reports the results. Sections 5 and 6 discuss the implications and the threats to validity, and Section 7

concludes. The supplementary material, linked under Data and Ethics, gives the prompts and additional results (Section S1), the perturbation test (Section S2) and further method and result details (Section S3).

2 Related Work

2.1 LLM-Based Gherkin and BDD Generation

Research on LLM-assisted BDD generation has typically emphasized either semantic resemblance or syntactic correctness; Table S5 of the supplementary material summarizes the five studies closest to ours. Rathnayake et al. [1] evaluated GPT-4, Claude 3 and Gemini on 500 user stories from one company and reported a BERTScore F1 of 91.16% for GPT-4. Fernandes et al. [4] found business-logic flaws, such as a reversed or omitted outcome, among the most frequent errors of seven LLMs. Karpurapu et al. [2] found that zero-shot outputs accounted for 89% of the observed Gherkin-lint syntax errors, but their validator was a linter rather than a runtime parser. In industrial workflows, all 50 Cypress test cases of Ferreira et al. [3] were valid TypeScript, yet only 60% reflected their Gherkin scenarios, and Fonseca et al. [5] found 93.3% of their generated scenarios syntactically valid by a manual keyword check. These single-organization or single-application evaluations show that LLMs can be integrated into practical workflows, but they make cross-domain generalization difficult to assess.

2.2 Test Generation and Semantic Evaluation

Broader software-test generation research shows that syntactic success and fault-revealing or semantic quality should not be conflated. Benchmarks exist for class-level test-case generation [6] and for test-case generation over generated program structures [7], and XUAT-Copilot uses LLM-based agents for user-acceptance testing [8]. For generated code, EvalPlus found pass@k up to 19.3–28.9% lower once the HumanEval tests were extended 80-fold [9], and CodeJudge shows that token- and embedding-based metrics fail on semantically equivalent code written differently [10]. Where an executable oracle exists it replaces resemblance [11,12,13]; Gherkin generation has no such oracle, because a valid scenario can still state the wrong business rule.

For comparison with a reference, sentence embeddings such as Sentence-BERT [16] provide a scalable alternative to manual labeling, but concrete names and values in raw Gherkin can dominate textual resemblance without representing domain intent. The present study therefore masks quoted values and numbers and keeps only keyword lines before embedding, treats the resulting skeleton similarity as a proxy rather than a measure of business correctness, and tests how well that proxy tracks business-logic defects (Section 4.4).

Reporting is a further concern. A review of 74 LLM studies in requirements engineering found that only 16% used public datasets and 39% did not state how prompts were selected [14]. Guidelines for empirical software-engineering studies

with LLMs require reporting the model version, configuration and date of use, publishing all prompts and explaining how few-shot examples were selected, and they name sensitivity to in-context examples as a validity threat [15].

2.3 Positioning of This Study

Table S5 compares this study with the five BDD-generation studies above on the criteria that bear on our contributions.

Contribution 1, the benchmark. The other studies with named sources each draw on one company, system or application, and two do not release their data (Table S5). Our 261 reference files are from three public projects in different domains and are released with their construction and a leakage measurement.

Contribution 2, the evaluation. Karpurapu et al. check syntax with a linter, and the other studies measure semantics with similarity metrics, ratings or an LLM validator. Fernandes et al. describe business-logic flaws qualitatively, but none of the five studies tests whether its semantic measure reacts to business-logic defects, and none varies which example is given in the prompt. Our defect audit and perturbation test show that files with high skeleton similarity can still violate their acceptance criteria, and our ablation shows that removing the example lowers skeleton similarity by more than repeated runs vary, while a different exemplar moves single files in both directions without a detectable average effect.

Contribution 3, the replication package. It contains the model identifier, prompts, scripts, API logs, generated outputs and all additional analyses, so that every number in this paper can be recomputed.

3 Methodology

Fig. S3 in the supplementary material shows the data flow and denominators. Full source-selection, prompt, logging and analysis details are in Supplementary Sections S1 and S3.

3.1 Dataset Construction

The dataset pairs 261 project-maintained Gherkin files with Connextra-style stories reverse-engineered from those files by GPT-5.4-mini at temperature 0. The prompt received at most the first 15,000 characters; this truncated 84 Fineract files. Generated files were scored against the full stored reference. Table 1 gives project sources and counts; Supplementary Section S3 documents the two collection batches, selection and extraction details.

3.2 Model, Prompts and Recording

All requests specified `gpt-5.4-mini-2026-03-17` [20] through the Chat Completions API [21], with temperature 0 and no output-token cap. The returned

Table 1. Dataset composition and sources. Commits are clone heads from 19 July 2026; .feature counts include all files at those commits.

Project	Domain	Branch, commit	.feature	Selected	Analysed
Sylius [17]	E-commerce	2.3, `9ab28ca`	839	100	99
Fineract [18]	Finance	develop, `5a93304`	147	100	99
Diaspora [19]	Social network	develop, `b7bf1dc`	71	61	60
Total			1,057	261	258

model name was recorded for C1, C2 and the rerun, but not original C0. C0 uses one same-project exemplar (ids 2, 102, 202); C1 removes the exemplar; C2 uses different same-project exemplars (ids 86, 119, 203). The primary C0 analysis excludes its three exemplars ($N = 258$); paired ablation uses 255 files that are non-exemplars in every condition. Full system messages, exemplar pairs, mapping and SHA-256 manifest are in Supplementary Section S1 (Table S1; Listings S1–S2) and the versioned replication package.

C0, C1 and C2 were rerun on 28 October 2026 with one script and logging path; four additional runs per condition used a fixed, project-stratified subset of 60 files. The same-time rerun also tested C0 with the proposal's 1,500-token cap (C0cap). The analysis plan and its hash were fixed before the rerun. C0 remains the primary result; a paired median change smaller than within-file run variation is not interpreted as an exemplar effect.

Failure policy. C0 used a 600-s client timeout and retried errors, including rate limits, after 60 s without a retry limit. C1/C2 allowed five attempts with waits of 5, 10, 20, 40 and 60 s; the rerun used a 600-s timeout, disabled client retries and allowed five attempts with 5, 10, 20 and 40-s waits. Empty outputs or terminal failures would count as parser FAILs and remain in the denominator, while empty outputs would be excluded from cosine analysis. All 261 C0 outputs and all C1/C2 and rerun requests succeeded on the first attempt; none was empty. Supplementary Section S3 records the run-specific policy and logs.

3.3 Measures and Defect Audit

Skeleton cosine. We retain Gherkin keyword lines, replace quoted strings and numbers with `<PARAM>` and `<NUM>`, and drop tags, tables, comments and other lines (Table S2). The skeletons are encoded with all-MiniLM-L6-v2 [16,22], mean-pooled and truncated at 256 word-piece tokens. The score is cosine similarity between generated and reference embeddings:

$$\mathrm{SkelCos}(g, r) = \frac{g \cdot r}{\|g\|\|r\|}. \tag{1}$$

Masking makes the score blind to values; truncation affects 67.4% of reference and 64.3% of generated skeletons. It measures structural resemblance, not business correctness. The preregistered 0.85 is an uncalibrated descriptive reference point, not an acceptance criterion; sensitivity is reported from 0.80 to 0.90 (Table S3).

Parser compliance. ESR is the share of generated files accepted by the `behave` 1.3.3 parser [23] without step definitions. A pass confirms valid Gherkin syntax, not step matching, execution or behavioral correctness. The 80% value is the prespecified RQ2 comparison rate.

Defect audit. We sampled 60 files with cosine ≥ 0.85 (20 per project) and labelled business-logic and temporal/causal defects against each story's criteria. OpenAI Codex drafted the labels; a second author reviewed them in one confirmation covering all 60 files. A first author independently labelled 15 files (6 Sylius, 3 Fineract, 6 Diaspora); agreement with the reviewed labels was Cohen's $\kappa = 0.571$ and 0.609 for the two facets [24], below the protocol's 0.7 target. The raters then held a per-file consensus round on those 15; the other 45 were not relabelled. The audit is exploratory. A separate keyword-order check and an eight-operator perturbation test on 120 scenarios (623 valid pairs) are detailed in Supplementary Section S2.

3.4 Statistical Analysis

Confirmatory tests use $\alpha = 0.05$, following guidance on statistical testing in software engineering [25]. For RQ1, the exact one-sided sign test tests whether the median exceeds 0.85; the median has a 95% percentile-bootstrap CI (10,000 resamples, seed 2026) [27], and the preregistered Wilcoxon test is a sensitivity analysis because the differences are skewed. We report $\delta = P(x > 0.85) - P(x < 0.85)$ with a CI. For RQ2, an exact binomial test [26] compares parser compliance with 80%; we report a Clopper–Pearson CI [28] and risk difference. Exploratory project comparisons use Kruskal–Wallis [29] and Holm-adjusted Mann–Whitney tests [30]; effect sizes are reported with CIs and no categorical magnitude labels. Leakage uses Spearman's ρ. Further test details are in Supplementary Section S3.

4 Results

4.1 RQ1: Structural Skeleton Resemblance

Among 258 non-exemplar files, the median cosine is 0.853 (95% bootstrap CI 0.844–0.859); 136 are above and 122 below 0.85. The exact one-sided sign test gives $p = 0.209$ and $\delta = 0.054$ (95% CI -0.072 to 0.179; Table S6 and Fig. S1). Differences are left-skewed (skewness -0.41, 95% CI -0.76 to -0.08; mean 0.848), so the Wilcoxon sensitivity test concerns the pseudo-median (Hodges–Lehmann estimate 0.850; $p = 0.474$). Including the three exemplars gives sign-test $p = 0.161$ and Wilcoxon $p = 0.355$. We fail to reject H_0 in all four analyses.

4.2 Results per Project

Sylius is the only project with a median above 0.85 (0.887); Fineract and Diaspora medians are 0.826 and 0.831 (Table 2; Fig. S2). Project medians differ

(Kruskal–Wallis $H = 60.87$, $p < 0.001$, $\eta^2_H = 0.231$, 95% CI 0.144–0.336); Holm-adjusted tests distinguish Sylius from both other projects ($p < 0.001$), but not Fineract from Diaspora ($p = 0.74$).

Truncated story input. The 84 Fineract files whose stories used only the first 15,000 characters do not score lower than the 15 non-truncated files (median 0.827 vs. 0.824; Mann–Whitney $p = 0.99$). Prefix rescoring changes one score by −0.004; the Fineract and pooled medians remain 0.826 and 0.853 (details in Supplementary Section S3).

Table 2. Skeleton cosine per project (C0, $N = 258$). Reported p-values are one-sided Wilcoxon tests against the 0.85 reference point.

Project	N	Median (IQR)	≥ 0.85 [95% CI]	$p_{>0.85}$	$p_{<0.85}$
Sylius	99	0.887 (0.076)	77.8% [68.3, 85.5]	< 0.001	1.000
Fineract	99	0.826 (0.062)	34.3% [25.1, 44.6]	1.000	< 0.001
Diaspora	60	0.831 (0.087)	41.7% [29.1, 55.1]	0.992	0.008
All	258	0.853 (0.085)	52.7% [46.4, 58.9]	0.474	0.526

4.3 Sensitivity to the Reference Point

The pooled median is significantly above candidate reference points through 0.84 (at 0.84, sign-test $p = 0.002$, Wilcoxon $p = 0.0097$), but not at 0.85 or higher (Table S3). Project rankings remain stable. The audit cannot calibrate the reference point: it samples only files at or above 0.85, and cosine does not distinguish defective from non-defective files within that range (Mann–Whitney $p = 0.21$).

4.4 Defects in High-Similarity Outputs

Exploratory audit. Of 60 files with cosine ≥ 0.85, 25 (41.7%, 95% CI [29.1, 55.1]) have a severity-2-or-3 defect under the reviewed labels: 6/20 Sylius, 16/20 Fineract and 3/20 Diaspora. Both facets flag the same files. The most frequent defects are wrong expected outcomes (20 files) and outcomes without a verifying Then (19). This project-balanced high-similarity sample does not estimate the corpus defect rate; Supplementary Section S3 gives an example.

Agreement and consensus. On 15 doubly labelled files, the independent rater agreed on 12 per facet; $\kappa = 0.571$ (95% CI 0.05–1.00) for business logic and 0.609 (0.24–1.00) for temporal/causal defects. After consensus, 6 files have business-logic and 8 temporal/causal defects, compared with 5 and 5 under reviewed labels. The other 45 files were not relabelled; applying consensus labels only to the 15 would yield 28/60 defective files (26 business-logic).

The keyword-order check flags 5 of 255 parseable generated files and 40 of 244 parseable references. In 623 valid perturbation pairs, no seeded defect lowers cosine below 0.8995; this scenario-level test does not estimate a whole-file false-negative rate (Section S2).

4.5 Lexical Leakage from Reverse Engineering

Median story–reference trigram overlap is 4.9% (Sylius 8.3%, Fineract 1.6%, Diaspora 5.8%). Overlap correlates with cosine in Sylius ($\rho = 0.29$, 95% CI 0.11–0.46, $p = 0.003$) and Diaspora (0.33, 0.08–0.55, $p = 0.009$), but not Fineract (0.11, −0.11–0.31, $p = 0.30$). The pooled $\rho = 0.43$ (95% CI 0.33–0.53) is inflated by project differences.

4.6 RQ2: Parser Compliance

Of 258 non-exemplar files, 255 pass (98.84%, 95% CI [96.64%, 99.76%], exact binomial $p < 0.001$; Table S4). The risk difference from 80% is +18.84 percentage points (95% CI [16.64, 19.76]). We reject H_0.

4.7 Prompt Ablation

The same-time rerun compares 255 common files (Table 3). Removing the exemplar (C1) lowers cosine in 193 files; the median paired difference is −0.030 (95% CI −0.038 to −0.021, Holm-adjusted $p < 0.001$), exceeding within-file SD. Changing the exemplar (C2) raises cosine in 140 files and lowers it in 115 (median paired difference +0.008, 95% CI −0.001 to 0.015; adjusted $p < 0.001$). This shift is below within-file SD and is not interpreted as a robust exemplar effect. The mean C2–C0 difference is +0.004 (95% CI −0.011 to 0.020), with per-file intervals on both sides of zero (Section 4.8). Parser counts are 255, 252 and 254; Holm-adjusted McNemar $p = 0.50$ and 1.0. Table S7 reports the original runs.

Table 3. Prompt ablation on 255 files that are non-exemplars in every condition, same-time rerun (28 October 2026). Table S7 gives original runs.

Condition	Median	≥ 0.85 [95% CI]	Sylius	Fineract	Diaspora	PASS
C0 (exemplars 2, 102, 202)	0.852	52.5% [46.2, 58.8]	0.892	0.829	0.829	255
C1 zero-shot	0.811	33.7% [27.9, 39.9]	0.872	0.779	0.830	252
C2 (exemplars 86, 119, 203)	0.873	63.5% [57.3, 69.4]	0.891	0.873	0.841	254

4.8 Run-to-Run Variation

Across 60 repeated files, ICC(1,1) is 0.921, 0.904 and 0.929 for C0/C1/C2; within-file SD is 0.020, 0.022 and 0.019. Parser outcomes changed in 0/2/2 files, while 19/13/14 files crossed 0.85 across runs. Between original and rerun C0, 45 files crossed the reference point, but the new median is 0.852 (sign-test $p = 0.247$) and all 258 parse. The per-file Welch CI for C2–C0 lies wholly above zero for 19 files, wholly below for 17 and includes zero for 24. Changing the exemplar moves individual files both ways, with no detectable average effect.

5 Discussion

5.1 Parser Compliance and Structural Resemblance

Most outputs parse (255/258), consistent with prior studies reporting high syntax validity [2,5,3]; the ablation shows this does not depend on retaining the exemplar. Structural resemblance varies by project: Sylius is above the reference point, while Fineract and Diaspora are below. Long, data-driven Fineract files are often truncated by the encoder, but the 84 reverse-engineered stories based on prefixes do not explain the project difference. These factors were not isolated experimentally. Cross-study similarity scores are contextual, not rankings, because the datasets, metrics and tests differ [1,4].

5.2 What Similarity Can and Cannot Establish

Skeleton cosine measures keyword structure, not behavior. In the exploratory audit, 25 of 60 high-scoring files contained a reviewed defect; cosine did not distinguish defective files, and the scenario-level perturbation test left scores at or above 0.8995. Masked values cannot affect the score by construction. Thus neither 0.85 nor another point in the tested 0.80–0.90 range certifies acceptance-criterion correctness. The weak-to-moderate within-project overlap correlations are consistent with some lexical leakage from reverse-engineered stories, but do not establish causation.

5.3 Practical Use

The evidence supports assisted drafting: a reviewer should check expected outcomes, conditions, data values and criterion coverage. The exploratory, project-stratified audit found reviewed defects in 16/20 Fineract files, compared with 6/20 Sylius and 3/20 Diaspora files; this does not estimate defect prevalence in the full corpus. Parser success and high similarity do not support autonomous acceptance or unreviewed CI use.

6 Threats to Validity

For each threat we report what we did and the residual risk.

6.1 Internal Validity

Reverse-engineered stories. *Mitigation:* we measured trigram overlap and prefix-story performance (Sections 4.5 and 4.2). *Residual risk:* overlap correlates with similarity in two projects, structural leakage is unmeasured, and performance may be overstated for natural requirements; no independently written backlog was available.

Exemplars, pilot and hosted model. *Mitigation:* prompt exemplars are excluded from the primary analysis; C0–C2 were rerun with interleaved requests,

calls were logged, and run variation and the planned 1,500-token C0cap condition were measured. *Residual risk:* the 30 pilot stories are in the evaluated set, C2 uses one random exemplar draw, nondeterminism remains, and hosted serving changes cannot be detected. Under C0cap, long outputs were cut and parser compliance fell to 92.6%; the uncapped RQ2 result does not apply to that condition.

6.2 Construct Validity

Similarity. *Mitigation:* we compared cosine with defect labels, seeded perturbations, project results and reference points from 0.80 to 0.90. *Residual risk:* masking hides values, tables are omitted, many skeletons exceed the 256-token limit, and cosine did not separate defective from non-defective audited files.

Defect labels. *Mitigation:* one independent rater labelled 15 files; we report both κ values with CIs and held the required consensus round. *Residual risk:* labels were AI-drafted and reviewed in one confirmation; agreement is below 0.7 and imprecise; the other 45 files were not relabelled. The audit is exploratory and limited to files scoring at least 0.85.

Parser and perturbation. *Mitigation:* parser compliance uses `behave`; non-parsing mutant pairs were excluded. *Residual risk:* mutants are dependent single scenarios and do not estimate whole-file sensitivity; parser success does not execute steps or establish behavior.

6.3 External and Conclusion Validity

Projects and reference point. *Mitigation:* we report per-project results and the 0.80–0.90 sensitivity analysis. *Residual risk:* three public English-language projects are not representative of all BDD practice; 0.85 is uncalibrated, and the RQ1 decision changes between 0.84 and 0.85. Planned-after-review analyses are exploratory and their p-values are not corrected.

The study had no independently written requirements, could not detect hosted-service changes, did not execute generated files against step definitions, and could not relabel the 45 files outside consensus. Conclusions are limited to reverse-engineered stories from these projects during the observed run window, with skeleton cosine treated only as a proxy.

7 Conclusion

We evaluated GPT-5.4-mini reconstructing 261 feature files from reverse-engineered stories. The main contribution is an auditable multi-domain evaluation that separates parser compliance from structural resemblance and shows, through exploratory defect auditing and perturbation, why embedding similarity cannot certify business correctness. For RQ1, the median was 0.853; the exact sign test did not show that it exceeded the descriptive 0.85 reference point ($p = 0.209$, $\delta = 0.054$). For RQ2, 255/258 files parsed (98.84%, $p < 0.001$). In the reviewed high-similarity sample, 25/60 files had a business-logic defect;

labels had below-target agreement, consensus covered 15 files, and 45 were not relabelled. This project-stratified exploratory result does not estimate defect prevalence in the full corpus. GPT-5.4-mini is therefore supported as a drafting aid with human review of outcomes, conditions, values and criterion coverage, not as an autonomous acceptance-test generator. Future work should use independently written requirements, independent criterion-level review or execution against step definitions, and retrieval-augmented or open-source baselines.

Acknowledgments. GPT-5.4-mini is the system under evaluation and was also used to write the user stories from the reference files (Section 3). In the revision, an AI coding assistant (OpenAI Codex) drafted the 60 defect-audit labels, which an author reviewed, and AI assistants (OpenAI Codex and Anthropic Claude) helped write analysis scripts and draft text; Claude also prepared reading aids for the 15-file rater (rubric notes, an acceptance-criteria map, and reading sheets and a glossary for Fineract) that contained no labels. NotebookLM was used to locate passages in the related-work sources; every quoted fact was checked against the source PDFs. All numbers were recomputed from the released data and scripts, and the authors take responsibility for the content.

Data and Ethics. This study involves no human participants; the empirical subjects are public software artifacts and model-generated Gherkin scenarios. Human-subjects review is therefore not applicable. The replication package contains the dataset, prompts, scripts, generated outputs, API logs, audit labels and additional analyses at https://github.com/simonhoang611/RT-SWT-004-Group4. The supplementary material is available as the supplementary material (PDF).

Disclosure of Interests. The authors declare no external funding and no conflicts of interest.

References

1. Rathnayake, A., Shahin, M., Abaei, G.: Behaviour Driven Development Scenario Generation with Large Language Models. arXiv:2603.04729 (2026). https://doi.org/10.48550/arXiv.2603.04729
2. Karpurapu, S., Myneni, S., Nettur, U., Gajja, L.S., Burke, D., Stiehm, T., Payne, J.: Comprehensive Evaluation and Insights Into the Use of Large Language Models in the Automation of Behavior-Driven Development Acceptance Test Formulation. IEEE Access 12, 58715–58721 (2024). https://doi.org/10.1109/ACCESS.2024.3391815
3. Ferreira, M., Viegas, L., Faria, J.P., Lima, B.: Acceptance Test Generation with Large Language Models: An Industrial Case Study. In: IEEE/ACM International Conference on Automation of Software Test (AST), pp. 1–11 (2025). https://doi.org/10.1109/AST66626.2025.00007
4. Fernandes, H., Perkusich, M., Albuquerque, D., Silva, I., Santos, D.F.S., Gorgônio, K., Perkusich, A.: A Comparative Study of LLMs for Gherkin Generation. In: Brazilian Symposium on Software Engineering (SBES), pp. 171–181 (2025). https://doi.org/10.5753/SBES.2025.9888
5. Fonseca, P.L., Lima, B., Faria, J.P.: Streamlining Acceptance Test Generation for Mobile Applications Through Large Language Models: An Industrial Case Study. In: 40th IEEE/ACM International Conference on Automated Software Engineering (ASE), pp. 3311–3321 (2025). https://doi.org/10.1109/ASE63991.2025.00273
6. Zhang, Q., Shang, Y., Fang, C., Gu, S., Yu, S., Zhou, J., Chen, Z.: TestBench: Evaluating Class-Level Test Case Generation Capability of Large Language Models. Frontiers of Computer Science 21(6), 2106202 (2027), published online 13 October 2025. https://journal.hep.com.cn/fcs/EN/10.1007/s11704-025-50078-9, last accessed 2026/10/28.
7. Chang, H.-F., Shokrolah Shirazi, M.: A Systematic Approach for Assessing Large Language Models' Test Case Generation Capability. Software 4(1), 5 (2025). https://doi.org/10.3390/software4010005
8. Wang, Z., Wang, W., Li, Z., Wang, L., Yi, C., Xu, X., Cao, L., Su, H., Chen, S., Zhou, J.: XUAT-Copilot: Multi-Agent Collaborative System for Automated User Acceptance Testing with Large Language Model. arXiv:2401.02705 (2024). https://doi.org/10.48550/arXiv.2401.02705
9. Liu, J., Xia, C.S., Wang, Y., Zhang, L.: Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. In: Advances in Neural Information Processing Systems 36 (NeurIPS 2023), pp. 21558–21572 (2023). https://doi.org/10.52202/075280-0943
10. Tong, W., Zhang, T.: CodeJudge: Evaluating Code Generation with Large Language Models. In: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp. 20032–20051 (2024). https://doi.org/10.18653/v1/2024.emnlp-main.1118
11. Kang, S., Yoon, J., Yoo, S.: Large Language Models are Few-shot Testers: Exploring LLM-based General Bug Reproduction. In: 45th IEEE/ACM International Conference on Software Engineering (ICSE), pp. 2312–2323 (2023). https://doi.org/10.1109/ICSE48619.2023.00194
12. Fakhoury, S., Naik, A., Sakkas, G., Chakraborty, S., Lahiri, S.K.: LLM-Based Test-Driven Interactive Code Generation: User Study and Empirical Evaluation. IEEE Transactions on Software Engineering 50(9), 2254–2268 (2024). https://doi.org/10.1109/TSE.2024.3428972

13. Harman, M., Ritchey, J., Harper, I., Sengupta, S., Mao, K., Gulati, A., Foster, C., Robert, H.: Mutation-Guided LLM-based Test Generation at Meta. In: Companion Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering (FSE Companion), pp. 180–191 (2025). https://doi.org/10.1145/3696630.3728544
14. Zadenoori, M.A., Dąbrowski, J., Alhoshan, W., Zhao, L., Ferrari, A.: Large Language Models (LLMs) for Requirements Engineering (RE): A Systematic Literature Review. arXiv:2509.11446 (2025). https://doi.org/10.48550/arXiv.2509.11446
15. Baltes, S., Angermeir, F., Arora, C., Muñoz Barón, M., Chen, C., Böhme, L., Calefato, F., Ernst, N., Falessi, D., Fitzgerald, B., Fucci, D., He, J., Treude, C., Kalinowski, M., Lambiase, S., Russo, D., Lungu, M., Martinez Montes, C., Prechelt, L., Ralph, P., van Tonder, R., Wagner, S.: Guidelines for Empirical Studies in Software Engineering involving Large Language Models. arXiv:2508.15503v8 (2026). https://doi.org/10.48550/arXiv.2508.15503
16. Reimers, N., Gurevych, I.: Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In: EMNLP-IJCNLP, pp. 3982–3992 (2019). https://doi.org/10.18653/v1/D19-1410
17. Sylius: Sylius, branch 2.3, commit 9ab28ca. https://github.com/Sylius/Sylius/tree/9ab28ca4b5ad5797216fe14e72e4d6cfe29c3fdb, last accessed 2026/10/28.
18. Apache Software Foundation: Apache Fineract, branch develop, commit 5a93304. https://github.com/apache/fineract/tree/5a93304677061501f5d0c243b8dbf84752e08b0b, last accessed 2026/10/28.
19. diaspora* contributors: diaspora*, branch develop, commit b7bf1dc. https://github.com/diaspora/diaspora/tree/b7bf1dc33566c036a3d788dd8876e97c30905842, last accessed 2026/10/28.
20. OpenAI: GPT-5.4 Mini model documentation. https://developers.openai.com/api/docs/models/gpt-5.4-mini, last accessed 2026/10/28.
21. OpenAI: Create chat completion, API reference. https://developers.openai.com/api/reference/resources/chat/subresources/completions/methods/create, last accessed 2026/10/28.
22. Sentence Transformers: all-MiniLM-L6-v2 model card, revision 1110a24. https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2/tree/1110a243fdf4706b3f48f1d95db1a4f5529b4d41, last accessed 2026/10/28.
23. behave developers: behave v1.3.3 release. https://github.com/behave/behave/releases/tag/v1.3.3, last accessed 2026/10/28.
24. Cohen, J.: A Coefficient of Agreement for Nominal Scales. Educational and Psychological Measurement 20(1), 37–46 (1960). https://doi.org/10.1177/001316446002000104
25. Arcuri, A., Briand, L.C.: A Practical Guide for Using Statistical Tests to Assess Randomized Algorithms in Software Engineering. In: 33rd International Conference on Software Engineering, pp. 1–10 (2011). https://doi.org/10.1145/1985793.1985795
26. Agresti, A.: Categorical Data Analysis, 2nd edn. John Wiley & Sons (2002). https://doi.org/10.1002/0471249688
27. Efron, B., Tibshirani, R.J.: An Introduction to the Bootstrap. Monographs on Statistics and Applied Probability, vol. 57. Chapman & Hall, New York (1993)

28. Clopper, C.J., Pearson, E.S.: The Use of Confidence or Fiducial Limits Illustrated in the Case of the Binomial. Biometrika 26(4), 404–413 (1934). https://doi.org/10.1093/biomet/26.4.404
29. Kruskal, W.H., Wallis, W.A.: Use of Ranks in One-Criterion Variance Analysis. Journal of the American Statistical Association 47(260), 583–621 (1952). https://doi.org/10.1080/01621459.1952.10483441
30. Holm, S.: A Simple Sequentially Rejective Multiple Test Procedure. Scandinavian Journal of Statistics 6(2), 65–70 (1979). https://www.jstor.org/stable/4615733